Cours jQuery complet

Pour comprendre jQuery de zéro, avec des exemples basés sur le projet Ecocert (CodeIgniter 4 + Bootstrap 5).

1. C'est quoi jQuery ?

jQuery est une bibliothèque JavaScript qui simplifie trois choses :

- **Sélectionner** des éléments de la page (un champ, une liste, un bouton…).
- **Agir** dessus (changer le texte, cacher, griser, ajouter une classe…).
- **Réagir** aux actions de l'utilisateur (clic, changement de liste, saisie au clavier…) et **dialoguer avec le serveur** (AJAX).

La règle d'or tient en une phrase :

```
$('sélecteur').action();
```

`$` veut dire « jQuery ». Entre parenthèses, on indique **quel élément**. Après le point, on indique **ce qu'on en fait**.

Chargement dans votre projet

Dans `entete.php`, vous chargez déjà jQuery :

```
<script src="https://code.jquery.com/jquery-4.0.0.js"></script>
```

Il doit être chargé **avant** votre propre code, sinon `$` n'existe pas encore.

Attendre que la page soit prête

Tout votre code doit être écrit dans :

```
$(function(){
    // votre code ici
});
```

Cette fonction ne s'exécute qu'une fois la page entièrement construite. Sans elle, jQuery chercherait des éléments qui n'existent pas encore.

2. Les sélecteurs

Les sélecteurs jQuery sont les mêmes que ceux du CSS.

Sélecteur	Ce qu'il cible	Exemple
`'#id'`	l'élément ayant cet `id`	`$('#inputPays')`
`'.classe'`	tous les éléments ayant cette classe	`$('.form-select')`
`'balise'`	toutes les balises de ce type	`$('select')`
`'a, b'`	plusieurs sélecteurs à la fois	`$('input, select')`
`'parent enfant'`	descendants (à n'importe quel niveau)	`$('form input')`
`'parent > enfant'`	enfants directs uniquement	`$('ul > li')`
`'[attr]'`	éléments ayant cet attribut	`$('[required]')`
`'[attr=val]'`	attribut égal à une valeur	`$('[type=text]')`
`':first'`, `':last'`	premier, dernier	`$('li:first')`
`':checked'`	cases cochées	`$('input:checked')`
`':selected'`	option sélectionnée	`$('#inputPays option:selected')`
`':disabled'`, `':enabled'`	champs grisés ou actifs	`$('input:disabled')`
`':visible'`, `':hidden'`	éléments visibles ou cachés	`$('div:hidden')`

À retenir : un `id` est unique dans la page, donc `$('#id')` renvoie au plus un élément. Une classe ou une balise peut en renvoyer plusieurs, et l'action s'applique alors à tous.

3. Lire et modifier le contenu

`.text()` : le texte brut

```
$('h1').text();                 // lit le texte
$('h1').text('Nouveau titre');  // remplace le texte
```

`.html()` : le contenu HTML

```
$('#liste').html('<p>Bonjour</p>');  // insère du HTML
```

Attention : n'utilisez jamais `.html()` avec du texte venant d'un utilisateur ou de la base de données, car il peut contenir du code malveillant. Pour afficher des données, préférez `.text()`.

`.val()` : la valeur d'un champ de formulaire

```
$('#inputVille').val();           // lit la valeur saisie
$('#inputVille').val('');         // vide le champ
$('#inputPays').val('France');    // sélectionne l'option France
```

C'est ce que vous avez utilisé pour tester le pays : `$('#inputPays').val() === 'France'`.

`.attr()` et `.prop()` : les attributs

```
$('img').attr('src');               // lit un attribut
$('img').attr('src', 'logo.png');   // modifie un attribut
$('a').attr({ href: '/accueil', title: 'Accueil' });  // plusieurs d'un coup
$('img').removeAttr('title');       // supprime l'attribut
```

Différence entre `.attr()` et `.prop()` :

- `.attr()` lit ce qui est écrit dans le HTML de départ.
- `.prop()` lit l'**état actuel** de l'élément.

Pour les cases à cocher et les champs grisés, utilisez toujours `.prop()` :

```
$('#maCase').prop('checked');            // true ou false
$('#maCase').prop('checked', true);      // coche la case
$('#inputDepartement').prop('disabled', true);   // grise le champ
```

`.data()` : des données attachées à un élément

En HTML : `<button data-id="12">`. En jQuery :

```
$('button').data('id');   // 12
```

Pratique pour garder un identifiant sans l'afficher.

4. Les classes et le style

```
$('#liste').addClass('fw-bold');       // ajoute une classe
$('#liste').removeClass('fw-bold');    // retire une classe
$('#liste').toggleClass('fw-bold');    // ajoute si absente, retire si
présente
$('#liste').hasClass('fw-bold');       // true ou false
```

Pour changer directement le style CSS :

```
$('#liste').css('color', 'red');                       // une propriété
$('#liste').css({ color: 'red', 'font-size': '20px' }); // plusieurs
$('#liste').css('color');                              // lit la valeur
```

Avec Bootstrap, préférez `addClass` / `removeClass` avec des classes Bootstrap (`d-none` , `text-danger` , `fw-bold`) plutôt que `.css()` .

5. Afficher, cacher, animer

Méthode	Effet
`.show()` / `.hide()`	affiche ou cache
`.toggle()`	bascule entre visible et caché
`.fadeIn()` / `.fadeOut()`	apparition ou disparition en fondu
`.fadeToggle()`	bascule en fondu
`.slideDown()` / `.slideUp()`	déroule ou enroule
`.slideToggle()`	bascule en glissement

On peut donner une durée en millisecondes, et une fonction à exécuter à la fin :

```
$('#message').fadeOut(500, function(){
    console.log('Message disparu');
});
```

6. Modifier la structure de la page (le DOM)

Méthode	Effet
`.append(x)`	ajoute `x` **à la fin** de l'intérieur de l'élément

Méthode	Effet
`.prepend(x)`	ajoute `x` **au début** de l'intérieur
`.after(x)`	ajoute `x` **juste après** l'élément
`.before(x)`	ajoute `x` **juste avant** l'élément
`.remove()`	supprime l'élément
`.empty()`	vide l'intérieur de l'élément (il reste en place)
`.clone()`	crée une copie de l'élément
`.replaceWith(x)`	remplace l'élément par `x`

Exemple : ajouter une option dans une liste.

```
$('#inputPays').append('<option>Belgique</option>');
```

Exemple : remplir une liste avec des données reçues.

```
$('#inputDepartement').empty();   // on vide d'abord
$.each(data, function(index, ligne){
    $('#inputDepartement').append($('<option>').text(ligne.nom_departement));
});
```

`$('<option>')` crée un nouvel élément. Avec `.text(...)`, le contenu est inséré en toute sécurité.

7. Se déplacer dans la page

Méthode	Résultat
`.parent()`	le parent direct
`.parents()`	tous les ancêtres
`.closest('sel')`	l'ancêtre le plus proche correspondant au sélecteur
`.children()`	les enfants directs
`.find('sel')`	les descendants correspondant au sélecteur
`.siblings()`	les frères et soeurs (même parent)
`.next()` / `.prev()`	le suivant / le précédent

Méthode	Résultat
`.first()` / `.last()`	le premier / le dernier d'une sélection
`.eq(n)`	l'élément numéro `n` (on compte à partir de 0)
`.filter('sel')`	garde ceux qui correspondent
`.not('sel')`	exclut ceux qui correspondent

Exemple :

```
$('form').find('input').val('');   // vide tous les champs input du formulaire
```

`.each()` : parcourir une sélection ou un tableau

```
$('input').each(function(index){
    console.log(index, $(this).val());
});

$.each(tableau, function(index, valeur){
    console.log(index, valeur);
});
```

8. Les événements

Un événement, c'est quelque chose que fait l'utilisateur. On écrit ce qui doit se passer à ce moment-là.

La méthode `.on()` (à privilégier)

```
$('#bouton').on('click', function(){
    alert('Cliqué !');
});
```

Elle fonctionne avec tous les événements. Les raccourcis comme `.click(...)` ou `.change(...)` existent encore, mais `.on()` est la forme standard.

Les événements les plus courants

Événement	Quand il se déclenche
`click`	clic sur l'élément

Événement	Quand il se déclenche
`dblclick`	double-clic
`change`	la valeur d'un champ a changé (liste, case, champ quittée)
`input`	à chaque modification du texte, en direct
`keyup` / `keydown`	touche relâchée / enfoncée
`focus` / `blur`	le champ reçoit / perd le curseur
`submit`	envoi d'un formulaire
`mouseenter` / `mouseleave`	la souris entre / sort de l'élément
`scroll`	défilement

`$(this)` : l'élément qui a déclenché l'événement

Dans la fonction, `this` désigne l'élément cliqué ou modifié. On écrit `$(this)` pour pouvoir utiliser les méthodes jQuery dessus :

```
$('#inputPays').on('change', function(){
    var pays = $(this).val();   // $(this) = #inputPays
    console.log(pays);
});
```

C'est ce que fait votre page des privilèges : `var leprivilege = $(this).val();`.

Empêcher le comportement par défaut

Le formulaire s'envoie et la page se recharge. Pour bloquer cela :

```
$('form').on('submit', function(event){
    event.preventDefault();
    // votre code (par exemple un appel AJAX)
});
```

Éléments créés après coup : la délégation

Si vous ajoutez un bouton avec jQuery après le chargement, un `.on('click')` posé au départ ne le voit pas. Il faut l'attacher à un parent qui, lui, existe déjà :

```
$('#liste').on('click', '.supprimer', function(){
    $(this).closest('li').remove();
});
```

Supprimer un événement

```
$('#bouton').off('click');
```

9. Les formulaires

```
$('#champ').val();                 // lire
$('#champ').val('abc');            // écrire
$('#champ').prop('disabled', true);   // griser
$('#champ').prop('disabled', false);  // réactiver
$('#case').prop('checked');           // case cochée ?
$('input[name=sexe]:checked').val();  // bouton radio choisi
$('#liste option:selected').text();   // texte (pas la valeur) de l'option
choisie
$('form').serialize();                // tous les champs sous forme
nom=valeur&nom=valeur
```

Rappel important : un champ `disabled` n'est **pas** envoyé avec le formulaire. Si vous voulez qu'il soit envoyé tout en étant non modifiable, utilisez `readonly` à la place :

```
$('#champ').prop('readonly', true);
```

10. AJAX : dialoguer avec le serveur sans recharger la page

AJAX sert à envoyer une demande au serveur et à recevoir une réponse, sans recharger la page.

`$.ajax()` : la forme complète

C'est celle que vous utilisez dans `v_liste_utilisateurs.php` :

```
$.ajax({
    url: "<?= site_url('listeUtilisateursPrivilege') ?>",   // où envoyer la
demande
    method: 'POST',                                         // GET ou POST
    data: { privilege: leprivilege },                       // ce qu'on
envoie
    dataType: 'json',                                       // format de la
réponse attendue
    success: function(data){                                // si tout va
bien
        // data contient la réponse du serveur
    },
    error: function(){                                      // si ça échoue
        alert('Les données ne sont pas disponibles ...');
    }
});
```

Les paramètres principaux de `$.ajax()`

Paramètre	Rôle
`url`	l'adresse appelée
`method` (ou `type`)	`'GET'` ou `'POST'`
`data`	les données envoyées (objet ou texte)
`dataType`	format de réponse attendu : `'json'`, `'html'`, `'text'`
`success`	fonction appelée en cas de succès
`error`	fonction appelée en cas d'échec
`complete`	fonction appelée dans tous les cas, à la fin
`timeout`	durée maximale d'attente, en millisecondes

Les raccourcis

```
$.get(url, data, function(reponse){ ... });
$.post(url, data, function(reponse){ ... });
$.getJSON(url, function(reponse){ ... });
```

Le côté serveur (CodeIgniter)

AJAX appelle une **route**, qui appelle une méthode de contrôleur, qui renvoie du JSON. Chez vous :

- route : `$routes->post('listeUtilisateursPrivilege', 'entrainement\ajax\AjaxUtilisateurController::utilisateurPrivilege', ['filter'=>'auth']);`
- contrôleur : récupère `$this->request->getVar('privilege')`, interroge le modèle, puis fait `echo json_encode($data);`

Le chemin complet est donc : **événement jQuery → `$.ajax` → route → contrôleur → modèle → JSON → `success` → mise à jour de la page.**

Quand utiliser AJAX ?

Seulement quand les données **dépendent d'un choix de l'utilisateur** et doivent être demandées au serveur au bon moment. Si les données sont déjà dans la page (comme votre liste de départements), du simple JavaScript suffit.

11. Votre cas : pays et département

```
$(function(){
    function majDepartement(){
        if ($('#inputPays').val() === 'France') {
            $('#inputDepartement').prop('disabled', false).val('Tous');
        } else {
            $('#inputDepartement').val('Hors France').prop('disabled', true);
        }
    }
    $('#inputPays').on('change', majDepartement);
    majDepartement();
});
```

Ligne par ligne :

1. `$(function(){ ... })` : on attend que la page soit prête.
2. On définit une fonction `majDepartement` qui contient la règle.
3. `$('#inputPays').val()` : on lit le pays choisi.
4. Si c'est la France, `.prop('disabled', false)` active la liste et `.val('Tous')` la remet sur « Tous ».
5. Sinon, `.val('Hors France')` choisit cette option et `.prop('disabled', true)` grise la liste.

6. `.on('change', majDepartement)` : on relance la règle à chaque changement de pays.
7. `majDepartement();` : on l'applique aussi au chargement de la page.

Le chaînage : on peut enchaîner plusieurs actions sur le même élément, comme dans `.prop('disabled', false).val('Tous')`. Chaque méthode renvoie l'élément, ce qui permet d'écrire la suivante à la suite.

12. Les pièges classiques

- **`$` n'est pas défini** : jQuery est chargé après votre script, ou pas chargé du tout. Vérifiez l'ordre dans `entete.php`.
- **Rien ne se passe** : le sélecteur ne correspond à aucun élément. Testez avec `console.log($('#monId').length)` : 0 veut dire que rien n'a été trouvé.
- **`.attr('checked')` ne marche pas** : utilisez `.prop('checked')`.
- **Le code s'exécute avant la page** : mettez-le dans `$(function(){ ... })`.
- **`this` ou `$(this)`** : `this` est un élément brut, `$(this)` est l'élément avec les méthodes jQuery. Pour appeler `.val()`, il faut `$(this)`.
- **Un `id` écrit deux fois** dans la page : jQuery ne prend que le premier.
- **Un bouton créé après coup ne réagit pas** : utilisez la délégation (`$('#parent').on('click', '.bouton', ...)`).
- **Un champ `disabled` n'est pas envoyé** par le formulaire.
- **POST avec CodeIgniter 4 et la protection CSRF** : si elle est activée, il faut envoyer le jeton avec la requête, sinon le serveur refuse l'appel.
- **jQuery 4** (version que vous chargez) a supprimé d'anciennes fonctions utilitaires comme `$.trim`, `$.isArray` ou `$.parseJSON`. Utilisez à la place les équivalents JavaScript natifs (`.trim()`, `Array.isArray()`, `JSON.parse()`).

13. Outil de débogage

Ouvrez la console du navigateur avec **F12**, onglet **Console**. Utilisez :

```
console.log($('#inputPays').val());
```

pour afficher une valeur et vérifier que votre code fait bien ce que vous pensez. Les erreurs JavaScript s'y affichent aussi, avec le numéro de ligne.

14. Mémo rapide

Je veux…	J'écris
lire la valeur d'un champ	`$('#id').val()`
changer la valeur d'un champ	`$('#id').val('x')`
changer un texte	`$('#id').text('x')`
griser un champ	`$('#id').prop('disabled', true)`
cacher / afficher	`$('#id').hide()` / `.show()`
ajouter une classe	`$('#id').addClass('classe')`
réagir à un clic	`$('#id').on('click', function(){ ... })`
réagir à un changement de liste	`$('#id').on('change', function(){ ... })`
ajouter un élément	`$('#id').append('<option>x</option>')`
vider un conteneur	`$('#id').empty()`
parcourir un tableau	`$.each(tableau, function(i, v){ ... })`
appeler le serveur	`$.ajax({ url, method, data, dataType, success, error })`

Pour s'entraîner

1. Affichez dans la console la valeur du pays à chaque changement.
2. Ajoutez un bouton « Effacer » qui vide le champ Ville avec `.val('')`.
3. Cachez le champ Département avec `.hide()` quand le pays n'est pas la France, au lieu de le griser.
4. Chargez les départements d'un pays en AJAX, avec une nouvelle route et une nouvelle méthode de contrôleur.

Documentation officielle (en anglais) : https://api.jquery.com